permissionlesstech / **bitchat**

Code | Issues 42 | Pull requests 116 | Discussions | Actions | More

Projects

bitchat

Public

Notifications | Fork 5.9k | ☆

About

bluetooth mesh chat, IRC vibes

bluetooth · bluetooth-le · decentralized · e2e-encryption · ios · macos · mesh-network · messaging · nostr · swift

Readme

Unlicense

Security policy

Activity | Custom properties

37.0k stars | **302** watching | **5.9k** forks

Report repository

100 Branches | **24** Tags | Go to file | Go to file | Code

callebtc Port the Android globe geohash picker to iOS (#1807) ✓ 04995f6 · last week

.github/workflows	ci: prevent manual source mani…	last month
Configs	Bump version to 1.8.0 (#1796)	last week
bitchat.xcodeproj	build(deps): bump swift-secp25…	last week
bitchat	Port the Android globe geohas…	last week
bitchatShareExtension	Require review before importin…	3 months ago
bitchatTests	Port the Android globe geohas…	last week
docs	docs: why rotating the service …	last month
localPackages	Fix geochat timeline scrolling a…	last week
	Automated update of relay data…	3 months ago

relays		
scripts	Validate relay-directory change…	last month
.gitattributes	Prevent Github Languages stat…	last year
.gitignore	Pin announce signing keys to s…	3 months ago
.periphery.baseline.json	Link layer slice 5: BLELinkEven…	3 months ago
.periphery.yml	Quality pass on the 1.7.0 batch…	3 months ago
.swiftlint.yml	Make SwiftLint blocking in CI n…	2 months ago
BRING_THE_NOISE.md	Perf: Add `final` to classes that …	last year
Justfile	Fix $$ escaping that broke ever…	3 months ago
LICENSE	Add public domain license and …	last year
PRIVACY_POLICY.md	Keep working when the networ…	3 months ago
Package.resolved	build(deps): bump swift-secp25…	last week
Package.swift	Port the Android globe geohas…	last week
README.md	chore: fix receive typo and refr…	3 months ago
SECURITY.md	Add a real security policy (#1489)	3 months ago
WHITEPAPER.md	docs: correct inaccurate privac…	3 months ago

README | Unlicense license | Security



bitchat

A decentralized peer-to-peer messaging app with dual transport architecture: local Bluetooth mesh networks for offline communication and internet-based Nostr protocol for global reach. No accounts, no phone numbers, no central servers. It's the side-groupchat.

bitchat.free

📲 App Store

📲 Play Store

Getting a copy you can trust

Install from the App Store, or build from source you have verified. A compiled build from anywhere else cannot be verified — see Verifying bitchat for how to check source against the per-release hash manifest, and for what to do if that is the only build you can get.

This matters more than it usually would: this repository has been the target of takedown demands, and when a repository or releases page disappears, mirrors appear that nobody can check.

License

This project is released into the public domain. See the LICENSE file for details.

Features

- **Dual Transport Architecture**: Bluetooth mesh for offline + Nostr protocol for internet-based messaging
- **Location-Based Channels**: Geographic chat rooms using geohash coordinates over global Nostr relays
- **Intelligent Message Routing**: Automatically chooses best transport (Bluetooth → Nostr fallback)
- **Decentralized Mesh Network**: Automatic peer discovery and multi-hop message relay over Bluetooth LE
- **Privacy First**: No accounts, no phone numbers, no servers. Note that the mesh does use a persistent per-device identifier derived from your identity key — see the whitepaper on identity and metadata for what a nearby radio can observe
- **Private Message End-to-End Encryption**: Noise Protocol for mesh, BitChat private envelopes for Nostr fallback
- **IRC-Style Commands**: Familiar `/slap`, `/msg`, `/who` style interface
- **Universal App**: Native support for iOS and macOS
- **Emergency Wipe**: Triple-tap to instantly clear all data

- **Performance Optimizations**: LZ4 message compression, adaptive battery modes, and optimized networking

Technical Architecture

BitChat uses a **hybrid messaging architecture** with two complementary transport layers:

Bluetooth Mesh Network (Offline)

- **Local Communication**: Direct peer-to-peer within Bluetooth range
- **Multi-hop Relay**: Messages route through nearby devices (max 7 hops)
- **No Internet Required**: Works completely offline in disaster scenarios
- **Noise Protocol Encryption**: End-to-end encryption, with forward secrecy for live sessions (store-and-forward mail is sealed without it — see the whitepaper)
- **Binary Protocol**: Compact packet format optimized for Bluetooth LE constraints
- **Automatic Discovery**: Peer discovery and connection management
- **Adaptive Power**: Battery-optimized duty cycling

Nostr Protocol (Internet)

- **Global Reach**: Connect with users worldwide via internet relays
- **Location Channels**: Geographic chat rooms using geohash coordinates
- **440+ Relay Network**: Distributed across the globe for reliability
- **BitChat Private Envelopes**: App-specific encrypted private messages over Nostr relays
- **Ephemeral Keys**: Fresh cryptographic identity per geohash area

BitChat's private-envelope format is proprietary and is **not** NIP-17, NIP-44, or NIP-59 compatible. It uses Nostr as a relay transport but only interoperates with BitChat clients: private payloads travel inside kind-1059 events whose `v2:`-prefixed content is a BitChat-specific XChaCha20-Poly1305 construction, not NIP-44 encryption.

Channel Types

`mesh #bluetooth`

- **Transport**: Bluetooth Low Energy mesh network
- **Scope**: Local devices within multi-hop range
- **Internet**: Not required
- **Use Case**: Offline communication, protests, disasters, remote areas

Location Channels (`block #dr5rsj7`, `neighborhood #dr5rs`, `country #dr`)

- **Transport**: Nostr protocol over internet

- **Scope**: Geographic areas defined by geohash precision
  - `block` (7 chars): City block level
  - `neighborhood` (6 chars): District/neighborhood
  - `city` (5 chars): City level
  - `province` (4 chars): State/province
  - `region` (2 chars): Country/large region
- **Internet**: Required (connects to Nostr relays)
- **Use Case**: Location-based community chat, local events, regional discussions

Direct Message Routing

Private messages use **intelligent transport selection**:

1. **Bluetooth First** (preferred when available)
   - Direct connection with established Noise session
   - Fastest and most private option
2. **Nostr Fallback** (when Bluetooth unavailable)
   - Uses recipient's Nostr public key
   - BitChat's app-specific private-envelope encryption
   - Routes through global relay network
3. **Smart Queuing** (when neither available)
   - Messages queued until transport becomes available
   - Automatic delivery when connection established

For detailed protocol documentation, see the Technical Whitepaper.

Setup

Option 1: Using Xcode

```
open bitchat.xcodeproj
```

For a signed device build, create your ignored local configuration and replace the example team ID with your Apple Developer Team ID:

```
cp Configs/Local.xcconfig.example Configs/Local.xcconfig
```

`Local.xcconfig.example` derives unique app and App Group identifiers from that team ID. The entitlement files already reference `$(APP_GROUP_ID)`, so tracked project or entitlement files do not need to be edited.

Useful command-line checks from the repository root:

```
# macOS Debug build without signing
xcodebuild -project bitchat.xcodeproj -scheme "bitchat (macOS)" \
  -configuration Debug CODE_SIGNING_ALLOWED=NO build

# Full SwiftPM test suite
swift test

# iOS simulator tests
xcodebuild -project bitchat.xcodeproj -scheme "bitchat (iOS)" \
  -sdk iphonesimulator \
  -destination 'platform=iOS Simulator,name=iPhone 17' test
```

If `iPhone 17` is unavailable, choose an installed simulator from:

```
xcodebuild -showdestinations -project bitchat.xcodeproj -scheme "bitchat (iOS)"
```

Option 2: Using `just`

```
brew install just
just check
just run
```

`just build` and `just run` use the current `bitchat (macOS)` scheme and keep Xcode output in the ignored `.DerivedData/` directory. They never patch source, project, configuration, or entitlement files.

`just clean` removes only `.DerivedData/` and `.build/`. It does not invoke Git or restore tracked files, so uncommitted work is preserved. `just test` runs the SwiftPM suite and `just test-ios` runs the iPhone 17 simulator suite.

Localization

- App localizations live in `bitchat/Localizable.xcstrings`.
- Share extension strings are separate in `bitchatShareExtension/Localization/Localizable.xcstrings`.

Releases 21

v1.8.0 Latest

last week

+ 20 releases

Contributors 57



+ 43 contributors

Languages



Swift 99.1% **Other** 0.9%